

Modul mahasiswa Lab 06 - Compose: API, PostgreSQL, Redis

Sesi RPS: 06 · **Jalur utama:** tiga service Docker Compose lokal · **Prasyarat untuk:** repo Lab 07 setelah praktik ini

Hasil belajar

Anda dapat membaca [compose.yaml](#), menjalankan stack tiga service, menjelaskan DNS berbasis nama service, mengamati cache MISS/HIT, membuktikan persistensi database melalui volume, dan menjalankan profile debug tanpa mengunggah password. API di [api/app/main.py](#) memakai PostgreSQL sebagai sumber data dan Redis sebagai cache daftar catatan.

Teori singkat. Compose membangun aplikasi dari beberapa service pada network bersama. Service `api` memakai hostname `db` dan `cache` di dalam network itu; browser host mengakses API melalui `127.0.0.1:8000`. `depends_on` dengan `service_healthy` menunggu healthcheck dependency sebelum API dimulai. `db_data` dan `redis_data` adalah named volume sehingga data tetap ada ketika container diganti. Secret password dibaca dari file yang dipasang pada `/run/secrets/db_password`.

Persiapan

Pastikan Docker Desktop/Engine dan Compose plugin berjalan, port host **8000** kosong, serta terminal berada di root repo Lab 06. Hentikan container Lab 05 sebelum memakai port yang sama. Siapkan file lokal berikut; file nyata sudah diabaikan oleh Git.

PowerShell:

```
Copy-Item .\secrets\db_password.txt.example .\secrets\db_password.txt
Copy-Item .\env.example .\env
docker compose config --quiet
```

Bash/WSL:

```
cp secrets/db_password.txt.example secrets/db_password.txt
cp .env.example .env
docker compose config --quiet
```

Lab 06 · file lokal dan validasi Compose

```
$ Test-Path .env; Test-Path secrets/db_password.txt
True
True
```

```
$ docker compose config --quiet
(tidak ada teks; exit code 0)
```

```
$ git check-ignore -v .env secrets/db_password.txt
.gitignore:1:.env*      .env
.gitignore:3:secrets/*.txt secrets/db_password.txt
```

Output aktual. File rahasia hanya dicek keberadaannya; isi password tidak ditampilkan.

Perintah: `Test-Path .env`, `Test-Path secrets/db_password.txt`, `docker compose config --quiet`, dan `git check-ignore -v .env secrets/db_password.txt`. *Fungsi:* membuktikan file lokal ada, Compose dapat membaca YAML, dan Git mengabaikan kredensial. *Cara kerja:* `config --quiet` menguji konfigurasi tanpa mencetak password; `check-ignore` menunjukkan aturan pengecualian. *Baca hasil:* dua `True`, `exit 0`, serta dua aturan `.gitignore`. Ini render output command aktual tanpa menampilkan isi password.

Jika file lokal sudah ada dari percobaan sebelumnya, jangan menimpa data tanpa memeriksanya. Password contoh hanya untuk laptop latihan; ganti sebelum dipakai di lingkungan lain. Jangan taruh password di laporan atau screenshot.

Praktik 1 - Menyalakan dan mengamati stack

```
docker compose up --build -d --wait
docker compose ps
docker compose logs --tail=30 api db cache
docker compose exec api getent hosts db cache
```

Lab 06 · DNS internal, log, dan health

```
$ docker compose exec -T api getent hosts db cache
```

```
172.29.0.3      db
172.29.0.2      cache
```

```
$ docker compose logs --tail=4 api db cache
```

```
api-1 | INFO: 127.0.0.1:41092 - "GET /health HTTP/1.1" 200 OK
cache-1 | 1:M 07 Oct 2026 13:48:45.027 * Background saving started by pid 4295
cache-1 | 4295:C 07 Oct 2026 13:48:45.050 * DB saved on disk
api-1 | INFO: 127.0.0.1:36096 - "GET /health HTTP/1.1" 200 OK
api-1 | INFO: 127.0.0.1:42968 - "GET /health HTTP/1.1" 200 OK
api-1 | INFO: 127.0.0.1:56434 - "GET /health HTTP/1.1" 200 OK
cache-1 | 4295:C 07 Oct 2026 13:48:45.055 * Fork CoW for RDB: current 0 MB, peak 0 MB, average 0 MB
cache-1 | 1:M 07 Oct 2026 13:48:45.128 * Background saving terminated with success
db-1 | 2026-10-07 12:53:44.719 UTC [27] LOG: checkpoint starting: time
db-1 | 2026-10-07 12:53:45.170 UTC [27] LOG: checkpoint complete: wrote 6 buffers (0.0%); 0 WAL file(s) added, 0
removed, 0 recycled; write=0.326 s, sync=0.053 s, total=0.452 s; sync files=5, longest=0.027 s, average=0.011 s; distance=1
kB, estimate=1 kB; lsn=0/196F7B8, redo lsn=0/196F780
db-1 | 2026-10-07 13:03:44.331 UTC [27] LOG: checkpoint starting: time
db-1 | 2026-10-07 13:03:44.712 UTC [27] LOG: checkpoint complete: wrote 4 buffers (0.0%); 0 WAL file(s) added, 0
removed, 0 recycled; write=0.313 s, sync=0.026 s, total=0.381 s; sync files=4, longest=0.019 s, average=0.007 s; distance=1
kB, estimate=1 kB; lsn=0/196FCD0, redo lsn=0/196FC98
```

```
$ curl.exe -s -i http://127.0.0.1:8000/health
```

```
HTTP/1.1 200 OK
date: Wed, 07 Oct 2026 15:09:11 GMT
server: uvicorn
content-length: 53
content-type: application/json

{"status":"ok","database":"postgres","cache":"redis"}
```

Output aktual dari stack Compose sehat. IP container dapat berubah setelah down/up.

Perintah: docker compose exec -T api getent hosts db cache, docker compose logs --tail=4 api db cache, dan curl.exe -s -i http://127.0.0.1:8000/health. Fungsi: memeriksa hostname internal, jejak proses, serta respons HTTP. Cara kerja: DNS Compose memberi alamat db/cache di network internal; API memeriksa kedua dependency pada /health. Baca hasil: dua alamat, log service, HTTP 200 dan JSON status=ok. Ini render output command aktual; IP container bisa berubah.

Untuk mengamati log saat permintaan HTTP masuk, jalankan `docker compose logs -f api` di terminal kedua, lalu hentikan mode ikuti log dengan **Ctrl+C**. Ini hanya menghentikan tampilan log, bukan container.

PowerShell: `Invoke-RestMethod http://127.0.0.1:8000/health`; Bash: `curl -fsS http://127.0.0.1:8000/health`.

Checkpoint A: api, db, cache sehat; /health mengembalikan status=ok, database=postgres, dan cache=redis. Jika status masih `starting`, tunggu healthcheck lalu ulangi. Screenshot ini diambil dari stack lokal yang sedang berjalan.

Lab 06: tiga service Compose sehat

```
docker compose ps --format ...
```

NAME	SERVICE	STATUS	PORTS
cloud-notes-lab06-api-1	api	Up 2 minutes (healthy)	127.0.0.1:8000->8000/tcp
cloud-notes-lab06-cache-1	cache	Up 2 minutes (healthy)	6379/tcp
cloud-notes-lab06-db-1	db	Up 2 minutes (healthy)	5432/tcp

Tiga service Compose dalam keadaan sehat

- **Langkah:** Jalankan `docker compose up --build -d --wait` lalu `docker compose ps` dari root repo Lab 06.
- **Fungsi:** Memeriksa prasyarat API, PostgreSQL, dan Redis sebelum demo catatan. **Cara kerja:** Compose membangun/menjalankan service, jaringan, volume, dan healthcheck; API bergantung pada DB/cache, sedangkan `--wait` menunggu semuanya siap. **Baca hasil:** Cari tiga service `healthy` dan port API 8000.

Containers Give feedback

Container CPU usage 19.30% / 2000% (20 CPUs available) Container memory usage 391.35MB / 15.12GB Show charts

Search cloud-notes-lab06 Only show running containers

	Name	Container ID	Image	Port(s)	CPU (%)	Memory usag...	Actions
☐	cloud-notes-lab				1.07%	101.45MB / 46.4	
☐	api-1	d38ee88081e4	cloud-note	8000:8000	0.36%	44.64MB / 15.48	
☐	db-1	ff206ab5e145	postgres:1		0.02%	53.37MB / 15.48	
☐	cache-1	14b1485af167	redis:7-alpi		0.69%	3.44MB / 15.48G	

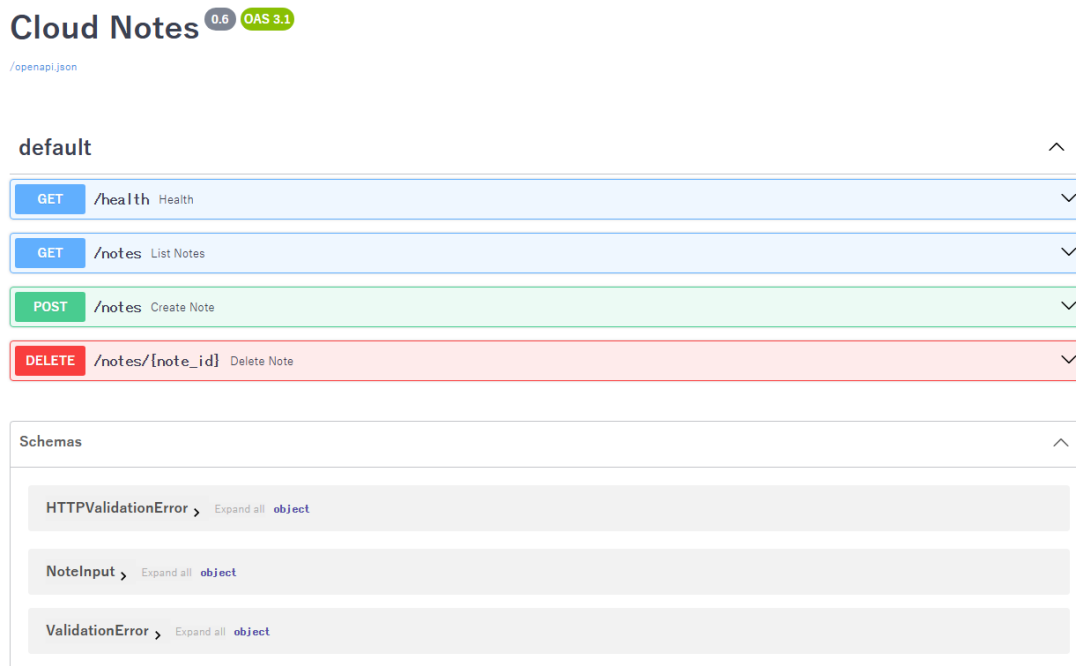
Showing 4 items

Engine running RAM 2.16 GB CPU 1.90% Disk: 347.08 GB used (limit 1006.85 GB) Terminal Update version

Langkah: Setelah `docker compose up --build -d --wait`, buka Docker Desktop > Containers dan cari project `cloud-notes-lab06`; bandingkan dengan `docker compose ps`. **Fungsi:** Memastikan tiga proses layanan benar-benar aktif secara visual. **Cara kerja:** Compose membuat container `api`, `db`, dan `cache` pada jaringan internal; hanya API memetakan port host `127.0.0.1:8000`. **Baca hasil:** Tiga titik hijau berarti container berjalan; cek tulisan `healthy` pada terminal untuk status healthcheck. DB 5432 dan Redis 6379 tidak diekspos ke host.

Gambar: `docker compose ps` dari stack lokal; tiga baris sehat menunjukkan API baru dimulai setelah database dan cache siap.

Buka <http://127.0.0.1:8000/docs> untuk melihat endpoint API versi **0.6**. Bandingkan dengan Lab 05: endpoint DELETE tersedia dan data catatan kini berasal dari PostgreSQL.



Dokumentasi OpenAPI Lab 06 yang berjalan di Docker Compose

- **Langkah:** Buka <http://127.0.0.1:8000/docs> setelah Compose sehat. **Fungsi:** Mengenali endpoint API versi Lab 06 yang memakai PostgreSQL dan Redis. **Cara kerja:** FastAPI menyajikan Swagger UI dari OpenAPI service `api` di stack Compose. **Baca hasil:** Cari `/health`, `/notes`, dan metode DELETE; uji GET health tanpa mengubah data.

Gambar: halaman Swagger dari API Compose yang benar-benar berjalan; gunakan GET `/health` untuk mencoba respons tanpa mengubah data.

Praktik 2 - Cache dan database

PowerShell:

```
$body = '{"title":"Compose","content":"Catatan saya bertahan di PostgreSQL"}'
Invoke-RestMethod http://127.0.0.1:8000/notes -Method Post -ContentType application/json -Body $body
(Invoke-WebRequest http://127.0.0.1:8000/notes -UseBasicParsing).Headers['X-Cache']
(Invoke-WebRequest http://127.0.0.1:8000/notes -UseBasicParsing).Headers['X-Cache']
```

Bash/WSL:

```
curl -fsS -X POST http://127.0.0.1:8000/notes -H 'Content-Type: application/json' -d
\'{"title":"Compose","content":"Catatan saya bertahan di PostgreSQL"}\'
curl -i http://127.0.0.1:8000/notes
curl -i http://127.0.0.1:8000/notes
```

Checkpoint B: header X-Cache bernilai MISS, lalu HIT jika panggilan kedua dilakukan dalam TTL 30 detik. POST berikutnya menghapus cache daftar sehingga GET selanjutnya kembali MISS. Jika lebih dari 30 detik berlalu, ulangi dua GET dengan cepat. Bukti contoh:

Lab 06: PostgreSQL dan cache Redis

POST /notes; GET /notes dua kali

```
POST /notes
{"id":1,"title":"Compose","content":"Data persisten di PostgreSQL","created_at":"2026-09-30T18:13:45.398658+00:00"}

GET /notes, request 1
X-Cache: MISS
[{"id":1,"title":"Compose","content":"Data persisten di PostgreSQL","created_at":"2026-09-30T18:13:45.398658+00:00"}]

GET /notes, request 2
X-Cache: HIT
[{"id":1,"title":"Compose","content":"Data persisten di PostgreSQL","created_at":"2026-09-30T18:13:45.398658+00:00"}]
```

Respons aktual API dengan cache MISS lalu HIT

- **Langkah:** Setelah POST catatan, panggil GET /notes dua kali berturut-turut dengan `curl -i`. **Fungsi:** Mengamati cache Redis pada request baca. **Cara kerja:** GET pertama mengambil PostgreSQL dan mengisi cache; GET kedua membaca cache sampai TTL habis. **Baca hasil:** Bandingkan header X-Cache: MISS lalu X-Cache: HIT dengan isi catatan yang tetap sama.

Periksa isi database dan coba restart API:

```
docker compose exec db psql -U clouduser -d cloudnotes -c 'SELECT id, title FROM notes;'
docker compose restart api
docker compose ps
```

Setelah API sehat lagi, PowerShell: `Invoke-RestMethod http://127.0.0.1:8000/notes`; Bash: `curl -fsS http://127.0.0.1:8000/notes`. **Checkpoint C:** catatan masih ada sesudah restart api.

Untuk menguji named volume lebih kuat, hentikan lalu hidupkan ulang stack:

```
docker compose --profile debug down
docker compose up -d --wait
docker compose exec db psql -U clouduser -d cloudnotes -c 'SELECT id, title FROM notes;'
```

Checkpoint D: catatan masih ada karena `down` tidak menghapus named volume. Jangan gunakan `down -v` selama data latihan masih diperlukan. Bila Anda lanjut ke Lab 07, biarkan stack menyala.

Cloud Services - hasil perintah lokal

Catatan bertahan di volume PostgreSQL

Output aktual dari sesi uji; ditata ulang agar mudah dibaca pada modul.

POST; SELECT; restart api; GET; down/up; SELECT

POST /notes

```
{ "id":1, "title":"Volume kelas", "content":"Bukti data persisten", "created_at":"2026-10-01T00:56:08.900569+00:00" }
```

SELECT dari PostgreSQL sebelum restart

id	title
1	Volume kelas

(1 row)

GET /notes setelah docker compose restart api

```
[{"id":1,"title":"Volume kelas","content":"Bukti data persisten","created_at":"2026-10-01T00:56:08.900569+00:00"}]
```

SELECT sesudah docker compose down / up

id	title
1	Volume kelas

(1 row)

Catatan yang sama terlihat sebelum dan sesudah restart serta down/up

- **Langkah:** POST catatan, query `SELECT id,title FROM notes`, `docker compose restart api`, lalu `down/up` dan query lagi. **Fungsi:** Menguji persistensi data lintas restart API dan stack. **Cara kerja:** PostgreSQL menulis ke volume Compose; restart proses API tidak menghapus volume. **Baca hasil:** Cocokkan ID/judul catatan sebelum dan sesudah; `down -v` baru menghapus volume latihan.

Gambar: output POST, query PostgreSQL, restart API, dan down/up yang benar-benar dijalankan. ID, judul, dan timestamp pada pekerjaan Anda dapat berbeda.

Profile debug dan analisis

Opsional, jalankan `docker compose --profile debug up -d adminer`, lalu buka `http://127.0.0.1:8081`. Pilih PostgreSQL; server db, pengguna `clouduser`, database `cloudnotes`, dan password dari file lokal. Jangan tampilkan password di screenshot. Adminer hanya perlu untuk pemeriksaan manual dan memakai port 8081.

Language: English PostgreSQL > db > cloudnotes > Schema: public clouduser Logout

Adminer 4.17.1 6.1.1

DB: cloudnotes
Schema: public

[SQL command](#) [Import](#)
[Export](#) [Create table](#)

[select notes](#)

Schema: public

[Alter schema](#) [Database schema](#)

Tables and views

Search data in tables (1)

☐	Table	Engine	Collation	Data Length?	Index Length?	Data Free	Auto Increment	Rows?	Comment?
☐	notes	table		16,384	16,384	?		?	~ -1
1 in total			en_US.utf8	16,384	16,384	0			

Selected (0)

Move to other database: public

[Create table](#) [Create view](#)

Routines

[Create function](#)

Sequences

[Create sequence](#)

User types

[Create type](#)

Halaman Adminer setelah login ke database cloudnotes

- **Langkah:** Jalankan `docker compose --profile debug up -d adminer`, buka port 8081, lalu login dengan akun lab. **Fungsi:** Melihat database lewat UI SQL tanpa mengubah service utama. **Cara kerja:** Adminer pada profile debug tersambung ke PostgreSQL lewat jaringan Compose. **Baca hasil:** Periksa nama database `cloudnotes` dan tabel `notes`; jangan tampilkan password pada bukti.

Gambar: tampilan Adminer dari service profile debug setelah login. Password tidak terlihat; pilih tabel `notes` untuk memeriksa catatan.

Language: English PostgreSQL > db > cloudnotes > public > Select: notes clouduser Logout

Adminer 4.17.1 6.1.1

DB: cloudnotes
Schema: public

[SQL command](#) [Import](#)
[Export](#) [Create table](#)

[select notes](#)

Select: notes

[Select data](#) [Show structure](#) [Alter table](#) [New item](#)

SELECT * FROM "notes" LIMIT 50 (0.001 s) [Edit](#)

☐	Modify	id	title	content	created_at
☐	edit	1	Volume kelas	Bukti data persisten	2026-10-01 00:56:08.900569+00

Whole result ☐ 1 row

[Import](#)

Baris catatan tersimpan pada tabel notes di Adminer

- **Langkah:** Di Adminer, pilih tabel `notes` lalu klik **Select data**. **Fungsi:** Membandingkan data database dengan respons API. **Cara kerja:** Adminer mengirim query `SELECT` ke PostgreSQL yang sama dengan sumber API. **Baca hasil:** Cocokkan ID/judul dengan hasil `GET /notes`; hindari tombol `Truncate` atau `Drop`.

Gambar: data yang sama seperti respons API tampil di PostgreSQL. Jangan gunakan tombol `Truncate/Drop` pada data yang masih diperlukan.

Jawab dalam laporan:

1. Mengapa kode API memakai `DB_HOST=db`, sedangkan browser memakai `127.0.0.1`?
2. Bukti apa yang membedakan hasil `restart api`, `down`, dan `down -v`? Jelaskan tanpa menjalankan `-v` bila data masih dipakai.
3. Apa hubungan `POST`, invalidasi cache, `MISS`, `HIT`, dan `TTL 30 detik`?
4. Apa yang terjadi bila db sehat tetapi cache belum sehat? Apa fungsi `depends_on` di konfigurasi ini?
5. Mengapa `.env.example` boleh ada di Git, sedangkan `.env` dan file password nyata harus tetap lokal?

Dari root repo, salin [template laporan](#) ke `hasil/lab06.md`: PowerShell `Copy-Item hasil/TEMPLATE_LAPORAN.md hasil/lab06.md`; Bash `cp hasil/TEMPLATE_LAPORAN.md hasil/lab06.md`. Sertakan screenshot **hasil Anda** di `hasil/bukti/`, hasil SQL tanpa password, dan penjelasan. Screenshot modul bukan bukti mahasiswa.

Git dan pembersihan

Jika Lab 07 akan segera dilakukan, biarkan stack hidup. Setelah selesai, dari root repo Lab 06 jalankan:

```
docker compose --profile debug down
```

Perintah ini juga menghentikan Adminer bila profile `debug` sempat aktif; volume tetap tersimpan. Hanya pada data latihan yang boleh dihapus, `docker compose --profile debug down -v` juga menghapus volume.

Dari root repo, ikuti [panduan Git](#):

```
git status --short
git check-ignore -v .env secrets/db_password.txt
git add compose.yaml api initdb secrets .env.example MODUL_MAHASISWA.md hasil/lab06.md hasil/bukti
git diff --cached --name-only
git diff --cached --check
git commit -m "lab06: Compose, cache, dan persistensi"
git push
```

Lab 06 · cek repo Git terbit

```
$ git remote -v
origin https://github.com/SeedFlora/meet6CloudService.git (fetch)
origin https://github.com/SeedFlora/meet6CloudService.git (push)
```

```
$ git status --short
?? screenshots/lab06_dns_log_health.png
?? screenshots/lab06_persiapan_config.png
```

```
$ git log -1 --oneline
cdaa37d Publish complete Lab 06 modules
```

```
$ git rev-parse HEAD; git ls-remote origin refs/heads/main
local: cdaa37dc29d14f2a86d82eb17e8dea90df104375
remote: cdaa37dc29d14f2a86d82eb17e8dea90df104375
Sama: True
```

Output aktual repo template SeedFlora. Mahasiswa melakukan commit/push pada repo pribadi.

SHA pada gambar adalah snapshot saat uji. Setelah modul diperbarui, jalankan ulang perintah untuk memeriksa commit terbaru.

Perintah: `git remote -v`, `git status --short`, `git log -1 --oneline`, `git rev-parse HEAD`, dan `git ls-remote origin refs/heads/main`. *Fungsi:* memeriksa tujuan serta hasil push modul dan kode. *Cara kerja:* cocokkan SHA lokal dengan branch main remote. *Baca hasil:* Sama: True pada repo pengajar; mahasiswa mengulangi pada repo pribadi setelah commit/push. Ini render output command aktual.

Jika folder bukti belum dibuat, hilangkan argumen tersebut. Sebelum commit, pastikan `.env` dan `secrets/db_password.txt` **tidak** staged; contoh `.env.example` dan `db_password.txt.example` boleh staged. Jika push pertama belum punya upstream, gunakan `git push -u origin main`.

Jika gagal: konflik port 8000 berarti API Lab 05 masih aktif. DB `password authentication failed` dapat terjadi bila password file diubah sementara volume lama menyimpan kredensial awal; gunakan password awal untuk volume itu atau minta dosen membantu reset data latihan. X-Cache dua kali MISS dapat terjadi bila jaraknya melewati TTL. Jalankan `docker compose logs --tail=50 api db cache` untuk membaca sebab, tanpa membagikan secret.

Pilihan cloud: stack utama ini memang untuk Docker lokal; deployment publik memerlukan desain jaringan, secret manager, dan layanan database/cache tersendiri. Tidak diperlukan untuk menyelesaikan checkpoint Lab 06.

Kunci tantangan: cache Redis berhenti saat layanan dipakai

Kasus kerja: operasi menerima laporan bahwa fitur daftar catatan lambat, tetapi data tidak boleh hilang. Tugas Anda membedakan gangguan cache dari gangguan database dan memulihkan service. Kunci berikut memakai stack Lab 06 yang sudah sehat dan berisi sedikitnya satu catatan. Jalankan di **root repo Lab 06**; jangan memakai `down -v` karena Lab 07 dapat menggunakan volume yang sama.

1. Catat keadaan sehat

PowerShell:

```
docker compose ps
Invoke-RestMethod http://127.0.0.1:8000/health
(Invoke-WebRequest http://127.0.0.1:8000/notes -UseBasicParsing).Headers['X-Cache']
```

Bash:

```
docker compose ps
curl -i http://127.0.0.1:8000/health
curl -i http://127.0.0.1:8000/notes
```

`/health` memeriksa PostgreSQL dan Redis. X-Cache mungkin MISS atau HIT tergantung request sebelumnya; keduanya normal pada keadaan sehat.

2. Simulasikan cache mati dan baca gejala

```
docker compose stop cache
docker compose ps
docker compose logs --tail=20 api cache
```

PowerShell:

```
try { Invoke-WebRequest http://127.0.0.1:8000/health -UseBasicParsing } catch {
    \[int]$_ .Exception.Response.StatusCode }
$r = Invoke-WebRequest http://127.0.0.1:8000/notes -UseBasicParsing
$r.StatusCode
$r.Headers['X-Cache']
$r.Content
```

Bash:

```
curl -s -o /dev/null -w '%{http_code}\n' http://127.0.0.1:8000/health
curl -i http://127.0.0.1:8000/notes
```

Hasil kunci: health menjadi **503** karena dependency cache tidak sehat. GET `/notes` tetap **200** dan memberi `X-Cache: MISS` karena kode memakai PostgreSQL sebagai sumber kebenaran jika Redis gagal. Request bisa melambat beberapa detik saat koneksi Redis gagal; pada uji lokal sekitar **4 detik**. Catatan tetap ada. Periksa `docker compose exec db psql -U clouduser -d cloudnotes -c 'SELECT id,title FROM notes;'` bila perlu membuktikan data di database.

3. Pulihkan cache

```
docker compose start cache
docker compose ps
```

Tunggu cache sehat. Di PowerShell, jalankan berurutan `Invoke-RestMethod http://127.0.0.1:8000/health` dan dua kali (`Invoke-WebRequest http://127.0.0.1:8000/notes -UseBasicParsing`).`Headers['X-Cache']`. Di Bash, gunakan `curl -i` untuk URL yang sama. Health kembali **200**; GET pertama setelah pemulihan biasanya `MISS` dan kedua `HIT` bila kurang dari TTL 30 detik. `docker compose restart api` dapat dipakai bila status health API belum pulih; ulangi sampai api healthy. Rekam `docker compose ps` serta kode HTTP sebelum/sesudah ke `hasil/lab06.md`.

Bonus - kunci cache berisi JSON rusak

Pada cache latihan milik stack ini saja, Anda dapat mensimulasikan entri daftar yang rusak. Jalankan saat semua service sehat:

```
docker compose exec -T cache redis-cli SET notes:list '{broken}'
```

Lalu panggil GET `/notes` dua kali cepat dengan perintah dari bagian sebelumnya. **Hasil kunci:** GET pertama tetap HTTP **200/MISS** dan membaca PostgreSQL; API membuang salinan cache yang tidak dapat diparse lalu mengisi cache baru. GET kedua HTTP **200/HIT**. `/health` tetap 200 karena Redis hidup; masalahnya adalah **isi cache**, bukan koneksi. Pada uji lokal kasus ini lulus dan catatan ID 1 tetap ada. Jangan uji ini pada Redis aplikasi lain.

```
Lab 06 - cache Redis berisi JSON rusak (uji aktual)
docker compose exec -T cache redis-cli SET notes:list "{broken}" -> OK
GET /notes -> HTTP 200, X-Cache: MISS
ID catatan tetap ada dari PostgreSQL
GET /notes ulang -> HTTP 200, X-Cache: HIT
GET /health -> HTTP 200
```

Transkrip Docker/API aktual ditata ulang; cache rusak diganti dari sumber PostgreSQL.

Perintah: `docker compose exec -T cache redis-cli SET notes:list '{broken}'`, lalu GET `/notes` dua kali. Gambar adalah transkrip output aktual yang ditata agar terbaca. Baris `MISS` membuktikan fallback ke database; `HIT` berikutnya membuktikan cache berhasil diisi ulang.

4. Kunci pertanyaan analisis

1. Nama `db` dan `cache` diterjemahkan oleh DNS jaringan Compose antar-container. Browser berada di host sehingga mengakses port API yang dipublish pada `127.0.0.1:8000`.
2. `restart api` mengganti proses API, bukan volume. `down` menghapus container/jaringan tetapi mempertahankan named volume. `down -v` juga menghapus named volume; gunakan hanya untuk data latihan yang memang boleh dibuang.
3. POST menyimpan catatan di PostgreSQL dan menghapus cache daftar. GET pertama membaca database dan memberi `MISS`, lalu mengisi Redis selama 30 detik. GET berikutnya membaca cache (`HIT`) sampai TTL habis atau

POST/DELETE berikutnya membatalkannya.

4. `depends_on: condition: service_healthy` menunda start API sampai DB dan cache sehat pada awal up; setelah stack berjalan, Compose tidak otomatis memulihkan proses API saat dependency tiba-tiba gagal. Health API dapat menjadi 503 saat cache mati, tetapi GET daftar masih punya jalur database.
5. `.env.example` dan file secret `.example` adalah template tanpa kredensial produksi. `.env` serta `secrets/db_password.txt` berisi nilai lokal dan sudah diabaikan Git; cek dengan `git check-ignore -v .env secrets/db_password.txt` sebelum commit.

5. Kunci cek Docker dan Git

`docker compose ps` menampilkan tiga service utama dalam kondisi healthy dan port API host 8000. `docker compose exec api getent hosts db cache` membuktikan service name dapat di-resolve di network internal. `docker compose exec db psql ...` membuktikan catatan ada di database. `git diff --cached --name-only` harus memperlihatkan kode, modul, dan laporan yang dipilih, tanpa password nyata. Sesudah demo, jalankan `docker compose --profile debug down` hanya bila Lab 07 tidak segera memakai stack ini.

```
Lab 06 - Compose/API aktual

docker compose up --build -d --wait -> 3 service healthy

GET /health -> HTTP 200, database=postgres, cache=redis

POST /notes -> HTTP 201, id=1, title=Audit kelas Lab06

GET /notes pertama -> HTTP 200, X-Cache: MISS

GET /notes kedua -> HTTP 200, X-Cache: HIT

POST title=" " -> HTTP 422
```

Output Docker/API aktual ditata ulang agar terbaca; secret tidak direkam.

Perintah: `docker compose up --build -d --wait`, POST satu catatan, lalu GET /notes dua kali. Cuplikan Docker/API aktual ditata agar terbaca: API, DB, dan Redis healthy; POST HTTP 201; header X-Cache berubah dari MISS ke HIT; judul hanya spasi ditolak HTTP 422.

Cloud Notes 0.6 OAS 3.1

/openapi.json

default ^

GET /health Health

GET /notes List Notes

POST /notes Create Note

DELETE /notes/{note_id} Delete Note

Schemas ^

HTTPValidationError > Expand all object

NoteInput > Expand all object

ValidationError > Expand all object

Perintah: buka `http://127.0.0.1:8000/docs` ketika `docker compose ps` sehat. Browser memuat Swagger UI dari API Compose, termasuk DELETE /notes/{note_id}. Gunakan ini untuk membandingkan kontrak API persisten dengan Lab 05.

```
Lab 06 - incident Redis dan persistensi, uji lokal aktual
docker compose stop cache
GET /health -> HTTP 503 (4.07 detik)
GET /notes -> HTTP 200, X-Cache: MISS (4.0 detik)
Catatan id=1 tetap ada di PostgreSQL
docker compose start cache -> healthy
docker compose restart api -> catatan id=1 tetap ada
docker compose --profile debug down; docker compose up -d --wait
Catatan id=1 tetap ada; SELECT count(*) = 3
docker compose ps -> api, db, cache healthy
```

Transkrip Docker/API aktual ditata ulang; tidak ada volume yang dihapus.

Perintah: docker compose stop cache, GET /health dan /notes, pulihkan cache, lalu docker compose restart api serta docker compose --profile debug down/docker compose up -d --wait. Gambar adalah transkrip Docker/API aktual yang ditata agar terbaca. Kode 503 menandai cache tidak sehat; GET tetap 200/MISS; ID catatan sama setelah restart dan down/up karena volume PostgreSQL tetap ada. Jangan menambah -v pada cleanup sebelum data latihan boleh dihapus.